AN INTERNSHIP REPORT ON

QUEUELESS

A Pre-Order and Pickup Backend for a College Canteen

Submitted in partial fulfilment of the requirements
for the award of the degree of

[Bachelor of Technology]

in

[Computer Science and Engineering]

Submitted by

[Your Name]

[Roll / Registration Number]

Under the guidance of

[Guide Name]

[Designation], [Department]

Internship completed through **Udemy**

Node.js, Express, MongoDB & More: The Complete Bootcamp

[Start date] to [End date]

College Logo

Department of [Computer Science and Engineering]

[COLLEGE / UNIVERSITY NAME]

[City, State] | Academic Year [2025-26]

Certificate

This is to certify that the internship report entitled **“QueueLess: A Pre-Order and Pickup Backend for a College Canteen”** is a bonafide record of the work carried out by **[Your Name]** (**[Roll Number]**) during the period [Start date] to [End date], in partial fulfilment of the requirements for the award of the degree of [Bachelor of Technology] in [Computer Science and Engineering] of [College / University Name].

The internship was completed online through the Udemy course *Node.js, Express, MongoDB & More: The Complete Bootcamp*. The work is satisfactory and was carried out under my supervision.

[Guide Name] [Designation] Internship Guide	[HOD Name] Head of the Department [Department]
Internal Examiner	External Examiner

Declaration

I, **[Your Name]**, hereby declare that the internship report entitled **“QueueLess: A Pre-Order and Pickup Backend for a College Canteen”** is a record of original work carried out by me during my online internship through the Udemy course *Node.js, Express, MongoDB & More: The Complete Bootcamp*, under the guidance of **[Guide Name]**.

The project described in this report was designed and implemented by me. To the best of my knowledge, this report has not been submitted to any other university or institution for the award of any degree or diploma.

Place: [City]

Date: [Date]

[Your Name]
[Roll Number]

Acknowledgement

I express my sincere gratitude to **[Guide Name]**, [Designation], for the guidance and encouragement given throughout this internship.

I am thankful to **[HOD Name]**, Head of the Department of [Department], for permitting me to undertake this internship in online mode, and to the management of **[College / University Name]** for providing the necessary facilities.

I thank **Jonas Schmedtmann**, instructor of the Udemy course *Node.js, Express, MongoDB & More: The Complete Bootcamp*, whose course formed the technical foundation of this work.

Finally, I thank my parents, family and friends for their constant support and motivation.

[Your Name]
[Roll Number]

Abstract

This report describes an online internship completed through the Udemy course *Node.js, Express, MongoDB & More: The Complete Bootcamp*, and QueueLess, the backend project built by applying what the course taught. The course covered server-side JavaScript with Node.js, REST API design with Express, data modelling with MongoDB and Mongoose, authentication with JSON Web Tokens, API security, file uploads, email and online payments.

QueueLess lets college students pre-order canteen food, pay online through Stripe Checkout and collect the order with a 4-digit pickup code, removing the need to stand in a queue. Canteen staff work from a live order queue and move each order through the states *paid*, *preparing*, *ready* and *collected*. Administrators manage the menu and users and view sales reports built with MongoDB aggregation pipelines. Students who have collected an item can rate it.

The backend exposes 5 resource groups under `/api/v1`, stores data in 4 MongoDB collections and enforces 3 user roles. An automated suite of 6 integration tests, written with Node's built-in test runner, passes in full. The internship provided practical, production-style experience in building secure, layered REST APIs.

Keywords: Node.js, Express, MongoDB, Mongoose, REST API, JWT, Stripe, MVC

Table of Contents

Certificate **2**

Declaration **3**

Acknowledgement **4**

Abstract **5**

1 Introduction **7**

1.1 About the Internship 7

1.2 Course Structure 7

1.3 Objectives 7

2 Technologies Learned **9**

2.1 Node.js 9

2.2 Express 9

2.3 MongoDB and Mongoose 9

2.4 Authentication and Security 9

2.5 Other Libraries and Tools 9

3 Project: QueueLess **11**

3.1 Problem Statement 11

3.2 Objectives 11

3.3 User Roles 11

3.4 System Architecture 11

3.5 Database Design 12

3.6 Modules and Key Features 13

3.7 API Endpoints 14

3.8 Implementation Highlights 15

4 Testing and Results **16**

5 Learning Outcomes and Conclusion **18**

References **20**

Annexure **20**

CHAPTER 1

Introduction

1.1 About the Internship

The internship was carried out online in a self-paced format through Udemy. The course chosen was *Node.js, Express, MongoDB & More: The Complete Bootcamp*, taught by Jonas Schmedtmann. It is a project-based course in back-end web development: students build a complete tour-booking application, *Natours*, step by step, learning each concept as it is needed.

1.2 Course Structure

Module	Topics covered
Node.js fundamentals	Modules, the file system, the event loop, streams, events, npm
Express	Routing, middleware, the request-response cycle, environment variables
MongoDB and Mongoose	CRUD, schemas, validation, middleware, virtuals, indexes
Advanced API features	Filtering, sorting, field limiting, pagination, aggregation pipelines
Error handling	Operational vs programming errors, a global error handler, async error wrappers
Authentication and authorization	JWT, password hashing, roles, password reset by email
Security	Rate limiting, secure HTTP headers, NoSQL-injection and XSS protection, parameter pollution
Data modelling	Embedding vs referencing, populate, nested routes, a handler factory
Uploads, email and payments	Multer, Sharp, Nodemailer, Stripe Checkout and webhooks
Deployment	Production settings, compression, CORS, graceful shutdown

Table 1.1: Modules of the Udemy bootcamp

1.3 Objectives

1. To learn how to build server-side applications with Node.js and Express.
2. To learn to model, query and aggregate data in MongoDB through Mongoose.
3. To apply secure authentication, authorization and API hardening.
4. To apply the course’s architecture to a new real-world problem.

5. To verify the result with automated tests and an API collection.

1.4 Organisation of the Report

Chapter 2 covers the technologies learned. Chapter 3 presents the QueueLess project. Chapter 4 describes testing and results. Chapter 5 presents the learning outcomes, challenges faced, conclusion and future scope.

CHAPTER 2

Technologies Learned

2.1 Node.js

Node.js is a JavaScript runtime built on Chrome's V8 engine. It runs on a single thread with an event loop and non-blocking I/O, which suits I/O-heavy web APIs. QueueLess targets Node.js 20 or later, uses the built-in `crypto` module for reset tokens and pickup codes, and the built-in `node:test` runner for tests.

2.2 Express

Express is a minimal web framework for Node.js. A request passes through a chain of middleware functions until one of them sends a response. The course covered routers, route parameters, chained route handlers and a global error-handling middleware, all of which are used in QueueLess.

2.3 MongoDB and Mongoose

MongoDB is a document database that stores JSON-like documents. Mongoose adds schemas, built-in and custom validators, document and query middleware (hooks), virtual fields, instance and static methods, and population of references. The course also covered aggregation pipelines, which QueueLess uses to compute ratings and sales reports.

2.4 Authentication and Security

- **bcryptjs** hashes passwords (cost factor 12) before they are stored.
- **jsonwebtoken** issues stateless JWTs, sent as a Bearer header or an HTTP-only cookie.
- **helmet** sets secure HTTP headers.
- **express-rate-limit** caps each IP at 100 API requests per hour.
- **express-mongo-sanitize** and **xss-clean** remove NoSQL-injection operators and HTML from input.
- **hpp** prevents HTTP parameter pollution, with a whitelist of filterable fields.
- **cors** limits cross-origin access to the configured front end.

2.5 Other Libraries and Tools

Library	Purpose in QueueLess

Multer	Receives uploaded images into memory
Sharp	Resizes images (600×400 menu photos) and converts them to JPEG
Nodemailer	Sends welcome, password-reset, order-confirmation and order-ready emails
Stripe	Hosted checkout, signed webhooks and refunds
slugify, validator	URL slugs for menu items; email validation
morgan, compression, dotenv	Request logging, gzip responses, configuration from `config.env`

Table 2.1: Supporting libraries

Development tools used: Visual Studio Code, Postman for manual API testing, MongoDB (Atlas / local server), npm and Git.

CHAPTER 3

Project: QueueLess

3.1 Problem Statement

College canteens are crowded during short breaks. Students spend much of a break queuing to order, pay and wait for food, while staff have no organised view of incoming orders. QueueLess moves ordering and payment online, so that a student arrives at the counter only when the food is ready.

3.2 Objectives

1. Let students browse the menu, order and pay online, and collect food with a pickup code.
2. Give staff a first-come-first-served queue and a controlled way to update order status.
3. Give administrators menu, user and sales management.
4. Keep the system secure, validated and testable.

3.3 User Roles

Role	Permissions
Student (default)	Browse available items, check out, view and cancel own orders, rate collected items
Staff	View the order queue, advance order status, verify pickup codes, mark items sold out
Admin	Create, update and delete menu items, manage users and roles, view sales reports, cancel orders

Table 3.1: User roles

3.4 System Architecture

QueueLess follows the Model-View-Controller (MVC) pattern taught in the course; the “view” is the JSON response, since the front end is a separate client. Every request passes through the layers shown in Figure 3.1.

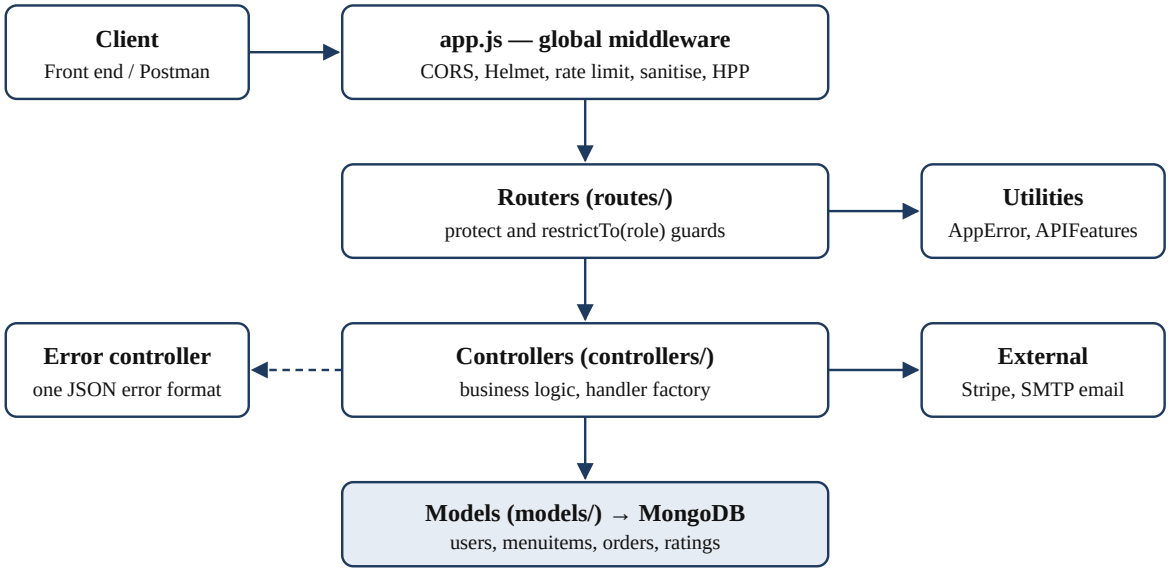


Figure 3.1: Layered architecture of QueueLess

1. **server.js** loads configuration, connects to MongoDB and handles uncaught exceptions, unhandled rejections and SIGTERM.
2. **app.js** applies global middleware: CORS, static files, Helmet, rate limiting, body parsing (10 kB limit), sanitising, HPP and compression.
3. **Routers** map URLs to handlers and apply the `protect` and `restrictTo` guards.
4. **Controllers** hold business logic; a handler factory provides generic CRUD handlers.
5. **Models** define Mongoose schemas, validation and hooks.
6. **Utilities** provide `AppError`, `catchAsync`, `APIFeatures`, `Email` and `filterObj`.

All errors reach a single global error controller, which converts database, JWT and upload errors into clear messages for the client.

3.5 Database Design

Collection	Key fields	Notable rules
users	name, email, photo, role, password, passwordChangedAt, reset token, active	Unique email; bcrypt hash on save; password hidden by default; soft delete via `active`
menuitems	name, slug, category, price, isVeg, image, isAvailable, prepTimeMinutes, ratingsAverage, ratingsQuantity	Category: snacks, meals, beverages or desserts; compound index on category and price; virtual `ratings`

orders	items[] (menuItem, name, price, quantity), totalAmount, status, pickupCode, paymentStatus, stripeSessionId, user	Status enum; unique Stripe session id; index on status and createdAt
ratings	rating (1–5), comment (max 300 chars), menuItem, user	One rating per user per item (unique compound index)

Table 3.2: MongoDB collections

Orders copy each item's name and price at the time of purchase, so later menu changes never alter past orders. Ratings use a static method with an aggregation pipeline to recompute an item's average and count after every save, update or delete.

3.6 Modules and Key Features

3.6.1 Authentication

Users can sign up, log in, log out, change their password and reset a forgotten password. The reset token is emailed and stored only as a SHA-256 hash that expires after 10 minutes. A JWT issued before the latest password change is rejected.

3.6.2 Menu

The list endpoint supports filtering, ranges (for example `price[lte]=50`), search, sorting, field limiting and pagination through the `APIFeatures` class. Students and guests see only available items. Staff may change only `isAvailable`; admins upload images, which Sharp resizes.

3.6.3 Checkout and Payment

A student sends a cart of up to 15 lines. The server checks opening hours in the canteen's time zone, item availability and quantities, then creates a Stripe Checkout session using prices read from the database, never from the client. After payment, Stripe calls a signed webhook, which creates the order with a random 4-digit pickup code and emails a confirmation. The handler is idempotent: a repeated webhook for the same session does not create a second order.

3.6.4 Order Queue and Status

Staff see paid and preparing orders oldest first, with pickup codes hidden. Status can move forward only one step at a time, as shown in Figure 3.2. Moving to *collected* requires the student's pickup code, and the student is emailed when the order becomes *ready*.

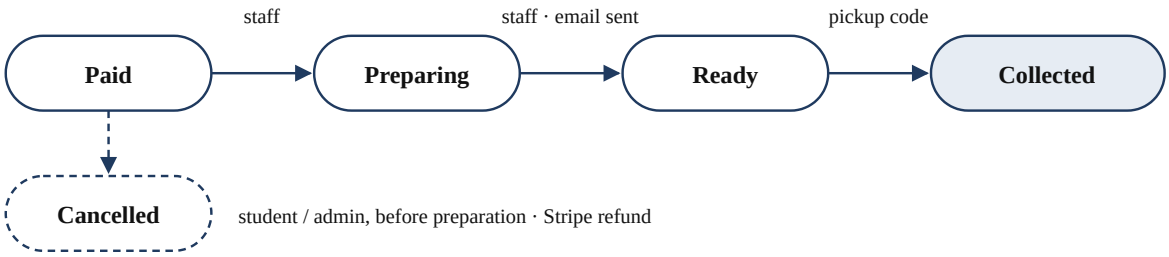


Figure 3.2: Order status lifecycle

3.6.5 Cancellation and Refunds

A student or admin may cancel an order only while it is still *paid*, before preparation starts; the payment is then refunded through Stripe.

3.6.6 Ratings

Ratings are a nested route under a menu item. A student may rate only items from an order they collected, once per item, and may edit only the score and comment of their own rating.

3.6.7 Sales Report

An admin requests `/api/v1/reports/sales?from=&to=` (default: the last 7 days). A single aggregation using `$facet` returns total revenue and orders, revenue by day, the top 10 items and orders by hour of the day.

3.7 API Endpoints

Method	Endpoint	Access
POST	/api/v1/users/signup, /login	Public
GET	/api/v1/users/logout	Public
POST, PATCH	/api/v1/users/forgotPassword, /resetPassword/:token	Public
GET, PATCH	/api/v1/users/me, /updateMe, /updateMyPassword	Logged in
GET, POST, PATCH, DELETE	/api/v1/users, /users/:id	Admin
GET	/api/v1/menu, /menu/:id	Public
POST, DELETE	/api/v1/menu, /menu/:id	Admin
PATCH	/api/v1/menu/:id	Staff (availability only), Admin
GET, POST	/api/v1/menu/:menuItemId/ratings	Public read; Student write

PATCH, DELETE	/api/v1/ratings/:id	Owner student; Admin may delete
POST	/api/v1/orders/checkout-session	Student
GET	/api/v1/orders/my-orders	Student
GET	/api/v1/orders/queue	Staff
GET	/api/v1/orders/:id	Owner, Staff, Admin
PATCH	/api/v1/orders/:id/status	Staff
PATCH	/api/v1/orders/:id/cancel	Student, Admin
GET	/api/v1/reports/sales	Admin
POST	/webhook-checkout	Stripe (signed)

Table 3.3: REST API endpoints

3.8 Implementation Highlights

- `catchAsync` wraps every async handler, so rejected promises reach the global error handler without try/catch blocks.
- `handlerFactory` provides `getAll`, `getOne`, `createOne`, `updateOne` and `deleteOne` for any model, removing repeated CRUD code.
- The Stripe webhook route is registered before `express.json()`, so its raw body can be verified against the signature.
- Sensitive data stays private: passwords are never returned, and pickup codes are shown only to the order's owner.
- `dev-data/data/import-dev-data.js` seeds 15 sample menu items and test users with `npm run import`.

CHAPTER 4

Testing and Results

4.1 Testing Approach

The integration tests in `test/api.test.js` use Node's built-in `node:test` runner and `fetch`, with no extra testing framework. Each run starts the real Express app on a random port, connects to a separate `queueless_test` database, seeds 3 menu items and 4 users (student, staff, admin and a second student), and drops the database at the end. Stripe webhooks are simulated with correctly signed test payloads, so the payment flow is tested without real payments.

Every endpoint was also tested manually using a Postman collection (`QueueLess.postman_collection.json`) organised into 7 folders: Auth, My account, Users (admin), Menu, Orders, Ratings and Reports (admin).

4.2 Test Cases

No.	Test case	What it verifies	Result
1	Login with a wrong password	Wrong credentials are rejected with HTTP 401	Pass
2	Menu visibility by role	Sold-out items are hidden from students but shown to staff	Pass
3	Menu filtering, ranges and search	Query-string features return the correct items	Pass
4	Staff can only toggle availability	Staff changing price or other fields is refused with HTTP 403	Pass
5	Students cannot reach the staff queue	Role guard returns HTTP 403	Pass
6	Full order flow	Signed webhook creates exactly one order; queue; status steps; wrong and correct pickup code; rating; sales report	Pass

Table 4.1: Automated test cases

4.3 Results

All 6 tests passed (`npm test`: 6 passed, 0 failed, about 3.7 seconds). During the test run the email transport deliberately points to a closed port; the logged "EMAIL ERROR" lines

confirm that a failed email is logged and never breaks an order.

4.4 Screenshots

[Insert the following screenshots before submission.]

- Figure 4.1: Terminal output of `npm test` showing 6 of 6 tests passing
- Figure 4.2: Postman login request and JWT response
- Figure 4.3: Menu list with filters in Postman
- Figure 4.4: Stripe Checkout page in test mode
- Figure 4.5: Staff order queue response
- Figure 4.6: Admin sales report response
- Figure 4.7: MongoDB Atlas / Compass view of the orders collection

CHAPTER 5

Learning Outcomes and Conclusion

5.1 Learning Outcomes

- Designing RESTful APIs with consistent URLs, status codes and JSON responses.
- Organising a Node.js codebase into routes, controllers, models and utilities.
- Modelling data in MongoDB and choosing between embedding and referencing.
- Writing Mongoose validators, hooks, virtuals and aggregation pipelines.
- Implementing JWT authentication, role-based authorization and secure password reset.
- Hardening an API against brute force, injection, XSS and parameter pollution.
- Integrating third-party services: Stripe payments with webhooks, and transactional email.
- Writing automated integration tests and testing APIs with Postman.

5.2 Challenges and Solutions

Challenge	Solution
The Stripe webhook signature failed after JSON parsing changed the body	Registered the webhook route with `express.raw()` before `express.json()`
Stripe may deliver the same webhook more than once	Checked for an existing order with that session id, backed by a unique index
Clients could tamper with prices sent in the cart	Read every price from the database when creating the checkout session
Orders could skip states or be collected by the wrong person	Allowed only the next status, and required the pickup code for collection
Opening hours depend on the canteen's time zone	Compared times using `toLocaleTimeString` in the configured time zone
A failed email could break an order request	Sent order emails in the background and logged failures

Table 5.1: Challenges faced and solutions

5.3 Conclusion

The internship met its objectives. The course gave a strong grounding in back-end development with Node.js, Express and MongoDB, and building QueueLess made it

possible to apply that knowledge to a real campus problem rather than only repeating the course project. The result is a secure, role-based API with online payments, an order queue, ratings and sales reporting, whose automated tests all pass.

5.4 Future Scope

1. A web or mobile front end for students, staff and admins.
2. Real-time queue updates with WebSockets instead of polling.
3. Estimated pickup times from item preparation times and queue length.
4. Push or SMS notifications when an order is ready.
5. Support for several canteens or outlets on one campus.
6. Cloud deployment with a CI pipeline running the test suite.

References

1. J. Schmedtmann, *Node.js, Express, MongoDB & More: The Complete Bootcamp*, Udemy. https://www.udemy.com/course/nodejs-express-mongodb-bootcamp/
2. Node.js Documentation. https://nodejs.org/docs/latest/api/
3. Express Documentation. https://expressjs.com/
4. MongoDB Manual. https://www.mongodb.com/docs/manual/
5. Mongoose Documentation. https://mongoosejs.com/docs/
6. Stripe Checkout and Webhooks Documentation. https://docs.stripe.com/payments/checkout
7. M. Jones, J. Bradley, N. Sakimura, “JSON Web Token (JWT)”, RFC 7519, IETF, 2015.
8. OWASP API Security Top 10. https://owasp.org/API-Security/

Annexure

- **Annexure A:** Udemy certificate of completion [attach]
- **Annexure B:** Project source code repository [link]